

**Fundamentals of Programming Mentorship:
C Coding Exercises – One Dimensional Arrays**

Fundamentals of Programming Mentorship:

C Coding Exercises – One Dimensional Arrays

Content

Arrays	1
One Dimensional Arrays	1
Easy Problems	2
Minimum and Maximum	2
Sort Check	2
Calculate the Mean	2
Reverse Array	3
Check if Palindrome	3
Sum of Even and Odd Values	3
Swapping Elements	4
Count Positive and Negative Numbers	4
Replace Negative Numbers with Zero	4
Finding the Second Largest Element	5
Check for X	5
Counting Specific Elements	5
Calculate the Median Version 1	6
Reverse Copy	7
Medium Problems	7
Number Tallying	7
Count Distinct Values	8
Find the Mode: Version 1	8
Longest Sub Array of 1's	8
Largest Difference	9
Cumulative Sum Array	9
Two Sum: Version 1	10
Array Concatenation	10
Set Union	11
Set Intersection	11
Fibonacci Array	12
Find the Missing Number in a Sorted Sequence Version 1	12
Left Rotate Array By One	12

Hard Problems	13
Find the Mode: Version 2	13
Find the Missing Number in a Sorted Sequence Version 1	13
Shift Elements in Array	13
Find the Missing Number in a Sequence Version 1	14
Find the Missing Number in a Sequence Version 2	14
Subarray of Length Three with Maximum Sum:	14
Merge Two Sorted Arrays.....	14
Sorting an Array	15
Reverse in Groups	15
Random Fibonacci Array (Past Lab Exercise).....	16
Move Zeroes to End Version 2.....	16
Insert Element to a Sorted Array	16
Calculate the Median Version 2	17
Swertres Lotto Version 1	18

Arrays

An array is a data structure that stores a collection of elements, typically of the same data type, in a contiguous block of memory. Each element in an array is accessible by its index, allowing for efficient retrieval and modification. Arrays are commonly used to store lists of values (such as numbers, characters, or objects) when the number of elements is known in advance.

Key Characteristics of Arrays:

- **Fixed Size:** Once an array is created, its size is set and cannot be changed.
- **Indexed Access:** Each element has an index, starting from 0, which allows for fast access.
- **Homogeneous Data:** Arrays usually store elements of the same data type.

One Dimensional Arrays

A one-dimensional array (1D array) is a data structure that stores a sequence of elements in a single, linear format. Each element in the array is accessible via a unique index, starting from 0 up to the array's length minus one.

Key Characteristics

- **Linear Structure:** Elements are stored in a straight line.
- **Index-Based Access:** Each element is accessed by its index, allowing fast retrieval.
- **Fixed Size:** In C, the size is set at creation and cannot be changed.

General Syntax:

```
data_type array_name[array_size];
```

Ex.

```
int arr[5]; //Declares an array of integers with 5 elements
int arr[5] = {1, 2, 3, 4, 5}; //Declares and initializes an array
int arr[] = {1, 2, 3, 4, 5}; //Size is inferred as 5
```

Accessing and Displaying Elements

```
int x = arr[0]; //Accesses the first element
int z = arr[1]; //accesses the second element
int i = 5;
printf("%d", arr[i]); //displays the fifth element
printf("%d", arr[i - 1]); //displays the fourth element
```

Easy Problems

Minimum and Maximum

Problem Description:

You are given an array of integers with a length of n . Your task is to determine and output both the minimum and maximum values present in the array.

Ex.

```
Enter size of array: 5
Enter values: 5 3 2 1 4
Minimum: 1
Maximum: 5
```

```
Enter size of array: 5
Enter values: -1 -2 0 3 -8
Minimum: -8
Maximum: 3
```

Sort Check

Problem Description:

Given an array of n length. Identify if the given array is sorted in ascending or descending order and if it is not sorted, simply output “*Not Sorted*”.

Ex.

```
Enter size of array: 5
Enter values: 1 2 3 4 5
The array is sorted in ascending order.
```

```
Enter size of array: 5
Enter values: -88 -100 -598 -999 -1000
The array is sorted in descending order.
```

Calculate the Mean

Problem Description:

Given an array of n length. Calculate its mean (average).

Ex.

```
Enter size of array: 5
Enter values: 1 2 3 4 5
Mean: 3.00
```

```
Enter size of array: 5
Enter values: -8 -9 -2 0 -100
Mean: -23.80
```

```
Enter size of array: 1
Enter values: 1
Mean: 1.00
```

Reverse Array

Problem Description:

Given an array of n length. Reverse the elements in the array.

Ex.

```
Enter size of array: 5
Enter values: 1 2 3 4 5
Reversed Copy: 5 4 3 2 1
```

```
Enter size of array: 5
Enter values: 1 2 3 2 1
Reversed Copy: 1 2 3 2 1
```

Check if Palindrome

Problem Description:

Given an array of n length. Determine if the elements in the array forms a palindrome.

Ex.

```
Enter size of array: 5
Enter values: 1 2 3 2 1
Array is a palindrome.
```

```
Enter size of array: 2
Enter values: 23
Array is not a palindrome.
```

```
Enter size of array: 1
Enter values: 9
Array is a palindrome.
```

Sum of Even and Odd Values

Problem Description:

Given an array of n length. Calculate the sum of all even numbers and the sum of all odd numbers in the array.

Ex.

```
Enter size of array: 5
Enter values: 1 2 3 4 5
Even Sum: 9
Odd Sum: 6
```

```
Enter size of array: 5
Enter values: -1 2 -3 -4 5
Even Sum: -2
Odd Sum: 1
```

```
Enter size of array: 1
Enter values: 1
Even Sum: 0
Odd Sum: 1
```

Swapping Elements

Problem Description:

Given an array of n length. Swap the first and last value in the array.

Ex.

Enter size of array: 5

Enter values: 1 1 2 3 5

After Swap: 5 1 2 3 1

Enter size of array: 6

Enter values: -1 -2 3 -4 5 -6

After Swap: -6 -2 3 -4 5 -1

Count Positive and Negative Numbers

Problem Description:

Given an array of n length. Count how many positive and negative numbers are in the array.

Ex.

Enter size of array: 5

Enter values: -1 2 -3 4 -5

Positive: 2

Negative: 3

Enter size of array: 3

Enter values: -1 2 1

Positive: 2

Negative: 1

Replace Negative Numbers with Zero

Problem Description:

Given an array of n length. Create a new array where all negative numbers are replaced with zero.

Ex.

Enter size of array: 5

Enter values: -1 2 -3 4 -5

Result: 0 2 0 4 0

Enter size of array: 5

Enter values: 1 2 3 4 5

Result: 1 2 3 4 5

Enter size of array: 5

Enter values: 0 0 0 -1 -2

Result: 0 0 0 0 0

Finding the Second Largest Element

Problem Description:

Given an array of n length. Identify the second largest unique number in the array and output both the number and the index.

Ex.

Enter size of array: 5

Enter values: -1 2 -3 4 -5

Second Largest: [3]: 2

Enter size of array: 5

Enter values: -1 0 -5 -8 - 6 -10

Second Largest: [0]: -1

Check for X

Problem Description:

Given an array of n length and a number x . Determine if the number x is present in the array.

Ex.

Enter size of array: 5

Enter values: 2 2 -3 2 -5

Enter x: 2

YES

Enter size of array: 5

Enter values: 1 2 3 4 5

Enter x: -2

NO

Counting Specific Elements

Problem Description:

Given an array of n length and a number x . Count occurrences of the number x in the array.

Ex.

Enter size of array: 5

Enter values: 2 2 -3 -2 -5

Enter x: 2

Occurrences: 2

Enter size of array: 5

Enter values: -1 2 3 4 -2

Enter x: 1

Occurrences: 0

Enter size of array: 3

Enter values: 1 1 1

Enter x: 1

Occurrences: 3

Calculate the Median Version 1

Problem Description:

The median is a measure of central tendency that represents the middle value of a sorted dataset, dividing it into two equal halves—50% of values are below the median, and 50% are above. It is particularly useful when outliers may skew the average (mean).

To calculate the median, the dataset must be sorted in ascending order. The calculation depends on the number of elements (n):

$$Med = \begin{cases} n \bmod 2 = 1, & \text{Value at position } \left(\frac{n+1}{2}\right) \\ n \bmod 2 = 0, & \frac{\text{Value at position } \left(\frac{n}{2}\right) + \text{Value at position } \left(\frac{n}{2} + 1\right)}{2} \end{cases}$$

where,

$Med = \text{median}$

$n = \text{number of elements}$

$\text{Value at position} = \text{the element in a given position}$

Note:

In this version, the array inputted must already be sorted.

Ex.

Enter size of array: 5

Enter values: 2 4 6 8 10

Median: 6

Enter size of array: 6

Enter values: 1 2 3 4 5 6

Median: 3.50

Enter size of array: 1

Enter values: 1

Median: 1

Enter size of array: 2

Enter values: 0 1

Median: 0.50

Reverse Copy

Problem Description:

Given an array of n length. Create a new array that is a reversed copy of the original array.

Ex.

```
Enter size of array: 5
Enter values: 1 2 3 4 5
Reversed Copy: 5 4 3 2 1
```

```
Enter size of array: 5
Enter values: 1 2 3 2 1
Reversed Copy: 1 2 3 2 1
```

Medium Problems

Number Tallying

Problem Description:

Given an array of n length that is initialized with value 0. Implement an input sequence where the user inputs value x ranging from 0 to $n - 1$ and the array is used to tally the how many times the value x is inputted by representing it as an index in the array. Display the array afterwards.

Note:

The input sequence is stopped once the user inputs a negative integer.

The input sequence is not the array.

The array is the tally of the input sequence.

Ex.

```
Enter size of array: 5
Enter values:
1
2
3
4
4
3
4
-1
Result: 0 1 1 2 3
```

```
Enter size of array: 3
Enter values:
0
2
2
-1
Result: 1 0 2
```

Count Distinct Values

Problem Description:

Distinct values are values within an array or any dataset that are considered to be unique, without any regard if they have duplicates within the array. Simply put, distinct values are all the unique values within the array.

Given an array of n length. Count all the distinct values in said array.

Ex.

```
Enter size of array: 8
Enter values: 1 3 2 2 3 3 5 4
Distinct Values: 5
```

```
Enter size of array: 5
Enter values: 0 0 0 1 2
Distinct Values: 3
```

Find the Mode: Version 1

Problem Description:

The **mode** is the value that appears most frequently in a dataset.

Given an array of length n . Identify the mode of said array.

Note:

If the array is multi-modal (multiple modes). Output only one of them.

If no value repeats, then the array has no mode.

Ex.

```
Enter size of array: 5
Enter values: 1 2 2 4 5
Mode: 2
```

```
Enter size of array: 5
Enter values: 1 1 2 2 5
Mode: 1
```

```
Enter size of array: 5
Enter values: -1 -2 -1 -1 5
Mode: -1
```

Longest Sub Array of 1's

Problem Description:

Given an array of n length with values 1's and 0's. Output the length of the longest sub array of 1's and the index where it starts.

Ex.

```
Enter size of array: 5
Enter values: 0 1 0 1 1
Longest Sub Array: 2 at index 3
```

```
Enter size of array: 1
Enter values: 1
Longest Sub Array: 1 at index 0
```

Largest Difference

Problem Description:

Given an array of n length. Calculate the largest difference of 2 values present within the array,

Ex. In printing the two values, it must also follow the order on how it was calculated.

Enter size of array: 8

Enter values: 1 1 2 2 3 3 4 4

Values: 4 and 1

Difference: 3

Enter size of array: 5

Enter values: -1 -2 -3 4 -5

Values: 4 and -5

Difference: 9

Cumulative Sum Array

Problem Description:

Given an array of n length. Create a new array that stores the cumulative sum of the original array.

Note: It is required that the cumulative sum be stored in a new array and not just immediately displayed.

Ex.

Enter size of array: 5

Enter values: 1 2 3 2 1

Cumulative Sum: 1 3 6 8 9

Enter size of array: 1

Enter values: -1

Cumulative Sum: -1

Two Sum: Version 1

Problem Description:

Given a sorted array of n length and a target value x . Display the two values present within the array that results to a sum of x . Additionally, display the indexes of the two values.

Note:

Display only one pair of values.

If there exists no pair, display "None".

Ex.

Enter size of array: 5

Enter values: 1 2 3 5 6

Enter target sum: 5

Result:

[1]: 2

[2]: 3

Enter size of array: 3

Enter values: 1 2 3

Enter target sum: 6

Result:

None

Array Concatenation

Problem Description:

Given two arrays of length n and m . Combine and store the two arrays in chronological order (based on which of the arrays was first inputted) in a new array of length $n + m$.

Ex.

Enter size of array A: 5

Enter values: 1 2 2 3 4

Enter size of array B: 6

Enter values: 3 4 4 5 6 6

Array C: 1 2 2 3 4 3 4 4 5 6 6

Enter size of array A: 0

Enter size of array B: 3

Enter values: 1 2 3

Array C: 1 2 3

Set Union

Problem Description:

A set union is the union of two or more sets. It is a set that contains all the elements of the given sets without its duplicates.

Ex.

Let:

$$A = \{1, 2, 2, 3, 4\}$$

$$B = \{3, 4, 4, 5, 6, 6\}$$

$$A \cup B = \{1, 2, 3, 4, 5, 6\}$$

Given two arrays of length n and m . Perform set union operation and store it in a new array of k length.

Ex.

Enter size of array A: 5

Enter values: 1 2 2 3 4

Enter size of array B: 6

Enter values: 3 4 4 5 6 6

A U B: 1 2 3 4 5 6

Enter size of array A: 0

Enter size of array B: 5

Enter values: 2 4 6 8 10

A U B: 2 4 6 8 10

Set Intersection

Problem Description:

A set intersection is the intersection of two or more sets. It is a set that contains the elements that are present in both of two given arrays, with no duplicates.

Ex.

Let:

$$A = \{1, 2, 2, 3, 4\}$$

$$B = \{3, 4, 4, 5, 6, 6\}$$

$$A \cap B = \{3, 4\}$$

Given two arrays of length n and m . Perform set intersection operation and store it in a new array of k length.

Ex.

Enter size of array A: 5

Enter values: 1 2 2 3 4

Enter size of array B: 6

Enter values: 3 4 4 5 6 6

A ∩ B: 3 4

Enter size of array A: 3

Enter values: 1 2 3

Enter size of array B: 2

Enter values: 4 5

A ∩ B:

Fibonacci Array

Problem Description:

Given a value n , create an array which stores the first n Fibonacci numbers.

Ex.

Enter number n : 5

Fibonacci Array: 0 1 1 2 3

Enter number n : 2

Fibonacci Array: 0 1

Find the Missing Number in a Sorted Sequence Version 1

Problem Description:

Given a list of numbers in a range with exactly one missing, find the missing number.

Ex.

Enter size of array: 5

Enter values: 1 1 3 4 5

Missing number: 2

Enter size of array: 3

Enter values: -1 1 2

Missing number: 0

Left Rotate Array By One

Problem Description:

Given an array of length n . Shift or rotate the elements on the array once to the left.

Ex.

Enter size of array: 5

Enter values: 1 1 3 4 5

Left Rotated by 1: 1 3 4 5 1

Enter size of array: 1

Enter values: -5

Left Rotated by 1: -5

Enter size of array: 2

Enter values: 2 9

Left Rotated by 1: 9 2

Enter size of array: 6

Enter values: 1 2 3 1 2 3

Left Rotated by 1: 2 3 1 2 3 1

Enter size of array: 3

Enter values: 1 0 1

Left Rotated by 1: 0 1 1

Hard Problems

Find the Mode: Version 2

Problem Description:

The **mode** is the value that appears most frequently in a dataset. Given an array of length n . Identify the mode of said array.

Note:

In this version, it is required that all modes are displayed.

If no value repeats, then the array has no mode.

Ex.

```
Enter size of array: 5
Enter values: 1 1 2 2 5
Modes: 1, 2
```

```
Enter size of array: 5
Enter values: 1 2 3 4 5
Modes: None
```

```
Enter size of array: 5
Enter values: -1 -2 -1 0 -2
Modes: -1, -2
```

Find the Missing Number in a Sorted Sequence Version 1

Problem Description:

Given a list of numbers in a range with missing numbers, find the missing numbers and create a new array that includes those numbers.

Ex.

```
Enter size of array: 7
Enter values: -2 0 3 3 4 5 7
Missing numbers: -1 1 2 6
-2 -1 0 1 2 3 3 4 5 6 7
```

Shift Elements in Array

Problem Description:

Given an array of n length and a value s . Create a new array where the elements stored have been shifted s places to the right from the original position.

Ex.

```
Enter size of array: 5
Enter values: 1 2 3 2 1
Enter shift value: 2
Shifted Array: 2 1 1 2 3
```

```
Enter size of array: 5
Enter values: 1 2 3 2 1
Enter shift value: 101
Shifted Array: 1 2 3 2 1
```

Find the Missing Number in a Sequence Version 1

Problem Description:

Given a list of numbers in a range with exactly one missing, find the missing number.

Ex.

```
Enter size of array: 5
Enter values: 1 4 3 5 1
Missing number: 2
```

```
Enter size of array: 3
Enter value: -2 1 0
Missing number: -1
```

Find the Missing Number in a Sequence Version 2

Problem Description:

Given a list of numbers in a range with exactly one missing, find the missing number.

Ex.

```
Enter size of array: 5
Enter values: 7 0 3 3 5 4
Missing number: 1 2 6
0 1 2 3 3 4 5 6 7
```

Subarray of Length Three with Maximum Sum:

Problem Description:

Given an array of length n . Find the contiguous subarray which has the largest sum given that the length of the subarray must be 3.

Ex.

```
Enter size of array: 5
Enter values: 7 0 3 3 5 4
Subarray: 3 5 4
Maximum Sum: 12
```

Merge Two Sorted Arrays

Problem Description:

Given two sorted arrays (ascending) arrays of length n and m . Create a new array with length $n + k$ which combines the two original arrays in ascending order.

Ex.

```
Enter size of array A: 5
Enter values: 1 3 5 9 10
Enter size of array B: 6
Enter values: 1 2 5 7 10 12
Merged and Sorted Array: 1 1 2 3 5 5 7 9 10 10 12
```

```
Enter size of array A: 3
Enter values: 3
Enter size of array B: 1 2 3
Enter values: 1 2 3
Merged and Sorted Array: 1 1 2 2 3 3
```

Sorting an Array

Problem Description:

Given an array of length n . Sort the array in ascending order.

Ex.

Enter size of array: 5

Enter values: 7 0 3 3 5 4

Ascending: 0 3 3 4 5 7

Enter size of array: 5

Enter values: -1 -90 100 5 96

Ascending: -90 -1 5 96 100

Enter size of array: 1

Enter values: 1

Ascending: 1

Reverse in Groups

Problem Description:

Given an array of length n . Reverses the elements in the array in groups of size k .

Ex.

Enter size of array: 6

Enter values: 1 1 3 4 5 1

Enter reverse size: 3

Result: 3 1 1 1 5 4

Enter size of array: 5

Enter values: 1 2 4 5 1

Enter reverse size: 1

Result: 1 2 4 5 1

Enter size of array: 3

Enter values: 1 3 2

Enter reverse size: 2

Result: 3 1 2

Enter size of array: 5

Enter values: 5 4 3 2 1

Enter reverse size: 5

Result: 1 2 3 4 5

Enter size of array: 8

Enter values: 2 4 6 8 1 3 5 7

Enter reverse size: 5

Enter result: 1 8 6 4 2 3 5 7

Random Fibonacci Array (Past Lab Exercise)

Problem Description:

Fibonacci sequence is a sequence in which each number is the sum of the two preceding ones usually with seed values 0 and 1.

0, 1, 1, 2, 3, 5, 8,

Declare an integer array with 15 elements and initialize it with the first 15 Fibonacci sequences excluding zero and 1 using iteration.

Declare another array with 50 elements and randomly initialize it with the index content from the Fibonacci array that you've constructed earlier. The code below would generate a random integer.

```
#include <stdlib.h> //library for rand()
#include <time.h> //library for time
int main ()
{
    time_t t;
    srand((unsigned) time(&t)); //seed random values with current time
    int x = rand(); //generate a random number and store it in x
    printf("%d\n", x); // print random number
}
```

Print the content of all the arrays that you've created.

Move Zeroes to End Version 2

Problem Description:

Given an array of length n . Rearrange an array so that all zeroes are moved to the end while maintaining the order of non-zero elements. It will not matter if the non-zero values are not arranged the same way as the original.

Ex.

Enter size of array: 6

Enter values: 0 1 0 4 5 1

Result: 1 4 5 1 0 0

Insert Element to a Sorted Array

Problem Description:

Given a sorted array of length n . Insert an element x at the correct position to keep it sorted. You are allowed to create a new array.

Ex.

Enter size of array: 6

Enter values: 1 2 5 6 9 10

Enter x: 3

Result: 1 2 3 5 6 9 10

Enter size of array: 1

Enter values: 0

Enter x: -2

Result: -2 0

Enter size of array: 3
 Enter values: 2 5 8
 Enter x: 10
 Result: 2 5 8 10

Calculate the Median Version 2

Problem Description:

The median is a measure of central tendency that represents the middle value of a sorted dataset, dividing it into two equal halves—50% of values are below the median, and 50% are above. It is particularly useful when outliers may skew the average (mean).

To calculate the median, the dataset must be sorted in ascending order. The calculation depends on the number of elements (n):

$$Med = \begin{cases} n \bmod 2 = 1, & \text{Value at position } \left(\frac{n+1}{2}\right) \\ n \bmod 2 = 0, & \frac{\text{Value at position } \left(\frac{n}{2}\right) + \text{Value at position } \left(\frac{n}{2} + 1\right)}{2} \end{cases}$$

where,
Med = median
n = number of elements
 Value at position = the element in a given position

Ex.

Enter size of array: 5
 Enter values: 10 4 8 6 2
 Median: 6

Enter size of array: 6
 Enter values: 1 5 3 6 2 4
 Median: 3.50

Enter size of array: 1
 Enter values: 1
 Median: 1

Enter size of array: 2
 Enter values: 0 1
 Median: 0.50

Swertres Lotto Version 1

Problem Description:

Swertres is a popular numbers game in the Philippines and it involves selecting a three-digit number combination that must match to another three-digit number drawn by the one that held the game.

Create a program that replicates the **Swertres Lotto Game**.

In order to generate some randomly drawn 3-digit numbers, implement the code below to randomly generate an integer.

```
#include <stdlib.h> //library for rand()
#include <time.h> //library for time
int main ()
{
    time_t t;
    srand((unsigned) time(&t)); //seed random values with current time
    int x = rand(); //generate a random number and store it in x
    printf("%d\n", x); // print random number
}
```

Note:

The problem refers to the drawn number as a three-digit number, however it must be implemented using an array and as such each digit must be stored in an index of an array.

The code above must only be able to generate numbers ranging from 1 – 9.

The numbers chosen by the player must match exactly to the numbers generated.

Ex.

```
SWERTRES LOTTO
ENTER 3 NUMBERS
1 2 3
RESULTS...
NUMBERS DRAWN: 3 1 2
YOU LOSE =(
```

```
SWERTRES LOTTO
ENTER 3 NUMBERS
2 4 6
RESULTS...
NUMBERS DRAWN: 2 4 6
YOU WIN =)
```